SCOA032: Artificial Intelligence

Master Study Guide (Chapters 1–8) | Examiner Pattern: Dr. V.R. Baloyi (2024–2026)

> **How to use this guide:** This document is a high-density, condensed master manual designed specifically for the SCOA032 examinations. It strips away fluff and focuses 100% on the theoretical definitions, algorithm traces, and applied scenarios repeatedly set by Dr. Baloyi and the departmental question bank.

Module Scope & High-Yield Exam Topics

Based on the exact 2024, 2025, and 2026 assessments, your preparation must center on the following rigorously tested paradigms:

Chapter	Core Theory (Definitions & Concepts)	Core Practical (Traces & Algorithms)
1. Roots & Scope	Agent types (Responsive vs. Proactive), Emergent intelligence themes, The Turing Test.	Applied Scenarios: Recommending Expert Systems (Geology) and Computer Vision (Hospital).
2. Predicate Calculus	Propositional laws, Soundness of inference (Modus Ponens/Tollens, Abduction).	Unification (MGU algorithm), Truth table proofs, Logic-Based Financial Advisor.
3. State Space Search	Goal-driven vs. Data-driven search justification, DFS vs. BFS comparison.	Backtracking algorithm traces (SL, NSL, DE lists), Nearest Neighbour (TSP).
4. Heuristic Search	Admissibility, Monotonicity, Informedness, Hill Climbing local maxima.	A* (Best-First Search) tracing with priority queues, Alpha-Beta Pruning in Game Trees.
6. Implementation	Production systems (Rules, Working Memory), Recognize-Act cycle, Blackboard architecture.	Forward Chaining diagramming, Knight's Tour Pattern Search trace.
7. Knowledge Rep.	Frames (Inheritance, Procedural attachments/demons), Experiencer links vs. relations.	Script writing (Restaurant/Exam), Drawing Conceptual Graphs from sentences.
8. Strong Method	Expert System architecture, 6 guidelines for appropriateness, Rule vs. Case vs. Model based.	Planning (STRIPS: Precondition, Add, Delete lists), Blocks World operator sequences.

Note on Algorithmic Tracing: Dr. Baloyi heavily penalizes "guessing" the final answer. For Backtracking, Best-First Search, and Alpha-Beta pruning, you must show the intermediate data structures (e.g., Working Memory updates, State Lists, or pruned branches) as explicitly demonstrated in the following chapters.

Chapter 1: AI Roots, Agents & Applications

1. The Turing Test (Imitation Game)

Proposed by Alan Turing to address the question "Can machines think?" It bypasses the philosophical debate of internal consciousness and focuses entirely on **observable behavior**.

- **Setup:** A human interrogator communicates with two entities via a teletype (text only). One is a human, the other is a computer.
- **Objective:** The interrogator must determine which is the computer. The computer's goal is to fool the interrogator.
- **Key Feature:** It prevents physical bias (no visual/auditory clues) and tests broad cognitive capacities (natural language, reasoning, knowledge recall).

> **Exam Focus: Agent-Oriented & Emergent Intelligence**
> *Tested in: 2025 Test 1, Ch 1 Recap*
> Modern AI views intelligence not as a single monolithic brain, but as emergent from simpler interacting entities. **The 5 main themes supporting this view are:**
>
> 1. **Autonomous or Semi-Autonomous:** Agents operate without direct human intervention and have control over their actions.
> 2. **Situated:** Agents exist in a complex, dynamic environment (they are not isolated in a lab).
> 3. **Interactional:** Agents interact with their environment and with other agents (often forming a society).
> 4. **Structured:** Agents have a defined internal architecture that processes inputs to outputs.
> 5. **Emergent:** Complex intelligent behavior emerges from the collective interactions of simple agents.

2. Types of Agents

Dr. Baloyi frequently asks for comparisons between specific agent architectures (e.g., 2026 Test 2):

- **Responsive (Reflex) Agent:** Operates strictly on a condition-action basis. It perceives the current state of the environment and reacts immediately according to pre-set rules. It has no memory of the past and does not plan for the future.
- **Proactive (Goal-Based) Agent:** Takes the initiative to achieve specific goals. It uses planning, considers the future consequences of its actions, and takes steps to bring about a desired state, rather than just reacting to the immediate present.

3. Defining AI

Artificial Intelligence is a multidisciplinary field of Computer Science concerned with the automation of intelligent behavior, aiming to create systems capable of problem-solving, learning, and reasoning.

Chapter 1: Applied AI Scenarios (Practical)

A major feature of recent SCOA032 papers is applied scenario questions where you must recommend an AI architecture, state its required elements, and note current limitations.

Scenario 1: Computer Vision (Mankweng Hospital)
"Mankweng Hospital requires your knowledge to help radiologists with computer vision systems..." (10 marks, 2024 Exam)

Recommendation: A deep learning-based Medical Image Analysis system (such as a Convolutional Neural Network / CNN) designed for anomaly detection (e.g., tumors, fractures) in X-rays and MRI scans.

Required Elements (Architecture):

1. **Image Acquisition System:** Sensors and scanners to digitize the radiology images.
2. **Preprocessing Module:** Enhances image contrast, removes noise, and standardizes image size.
3. **Feature Extraction & Classification Engine (Inference Engine):** The neural network that identifies patterns and classifies tissue as normal or abnormal.
4. **Knowledge Base:** A massive dataset of labeled historical medical images used to train the system.
5. **User Interface:** A dashboard for the radiologist to review highlighted anomalies and confirm diagnoses.

Current Deficits / Limitations: High dependency on massive amounts of perfectly labeled training data; "Black Box" nature (lack of explainability for why a decision was made); susceptibility to image noise; and inability to replace human clinical judgment in ambiguous cases.

Scenario 2: Expert System (Polokwane Geology Company)
"A geology company in Polokwane requires your expertise to locate mineral deposits..."

Recommendation: A Rule-Based Expert System (similar to PROSPECTOR) for geological data interpretation and mineral exploration.

Required Elements (Architecture):

1. **Knowledge Base:** Domain-specific rules provided by expert geologists (e.g., IF soil has high copper AND fault line is present, THEN high probability of ore).
2. **Inference Engine:** The logic controller that applies the rules to new survey data using forward or backward chaining.
3. **Working Memory:** Holds the specific data for the current geological site being evaluated.
4. **Explanation Subsystem:** Explains to the user *why* a certain location was recommended or *how* a conclusion was reached.

Current Deficits: The "Knowledge Acquisition Bottleneck" (hard to extract tacit knowledge from human experts); inability to learn or adapt automatically to new types of geological formations; fragile when dealing with data outside its narrow domain (the "brittleness" problem).

Chapter 2: Propositional Logic & Truth Tables

Propositional calculus uses symbols (P, Q, R) to represent facts that are either True (T) or False (F). You must master the logical connectives: Negation (¬), Conjunction (∧), Disjunction (∨), Implication (→), and Equivalence (↔).

Truth Table Proofs (Common Exam Question)
Prove that: $P \leftrightarrow Q \equiv (P \vee Q) \rightarrow (P \wedge Q)$ (2026 Test 1, 6 marks)

P	Q	P ↔ Q	(P ∨ Q)	(P ∧ Q)	(P ∨ Q) → (P ∧ Q)
T	T	**T**	T	T	**T**
T	F	**F**	T	F	**F**
F	T	**F**	T	F	**F**
F	F	**T**	F	F	**T** (Since F → F is True)

Conclusion: Since column 3 and column 6 are identical for all truth assignments, the statements are logically equivalent.

Soundness of Inference Rules
An inference rule is **sound** if it never derives a false conclusion from true premises.

1. Modus Ponens (Sound): From P → Q and P, infer Q.
2. Modus Tollens (Sound): From P → Q and ¬Q, infer ¬P.
3. Abduction (NOT Sound): From P → Q and Q, infer P.
Proof Abduction is not sound: If P = "It is raining" and Q = "The grass is wet". P → Q is true. If we observe Q (grass is wet), Abduction infers P (it is raining). However, the grass could be wet because of a sprinkler. Thus, Abduction can derive a false conclusion from true premises.

Translation into Propositional Logic

"I will pass Monday's test only if I attend classes or consults after each class." (2025 Test 1)
Step 1: Identify propositions.
P = I will pass Monday's test. Q = I attend classes. R = I attend consults.
Step 2: Translate. The phrase "only if" translates directly to implication (→).
Answer: `P → (Q ∨ R)`

Chapter 2: Predicate Calculus & Unification

While propositional logic handles whole sentences, Predicate Calculus allows us to look inside sentences using predicates, variables, and quantifiers (∀ for ALL, ∃ for EXISTS).

The Unification Algorithm

Unification is the process of finding a set of substitutions for variables that makes two predicate calculus expressions identical. This set is called the **Most General Unifier (MGU)**.

- Constants (a, b) can only bind to variables (X, Y).
- Constants cannot bind to other constants (e.g., `a` cannot unify with `b`).
- A variable cannot bind to a function that contains itself (e.g., `X` cannot unify with `father(X)`).

Unification Example 1 (2023 Exam)
Attempt to unify: p(X, a, Y, W) and p(Z, Z, b, C). Show the MGU or explain why they fail.

Step-by-step trace:

1. Predicates match (`p` = `p`). Arity matches (4 arguments).
2. Arg 1: Unify `X` and `Z`. Substitution: `{Z/X}`. Expressions become: `p(Z, a, Y, W)` and `p(Z, Z, b, C)`.
3. Arg 2: Unify `a` and `Z`. Substitution: `{a/Z}`. *(Apply to whole expression)*. Expressions become: `p(a, a, Y, W)` and `p(a, a, b, C)`.
4. Arg 3: Unify `Y` and `b`. Substitution: `{b/Y}`. Expressions become: `p(a, a, b, W)` and `p(a, a, b, C)`.
5. Arg 4: Unify `W` and `C`. Substitution: `{C/W}`.

Result: They unify. MGU = `{a/X, a/Z, b/Y, C/W}`.

Unification Example 2 (Failure)
Attempt to unify: p(A, B) and p(m, C) where A and B are variables, m is a constant. (2022 Supp)

1. Arg 1: Unify `A` with `m`. Substitution: `{m/A}`.
2. Arg 2: Unify `B` with `C`. Substitution: `{C/B}`.
MGU: `{m/A, C/B}`.

What if we had p(X, a) and p(b, Y)?
Arg 1 binds `{b/X}`. Arg 2 binds `{a/Y}`. MGU = `{b/X, a/Y}`.
What if we had p(a, X) and p(b, Y)?
Arg 1 attempts to unify constant `a` with constant `b`. **FAIL.** Constants cannot unify with different constants.

Chapter 2: The Logic-Based Financial Advisor

This is a 6-to-12 mark staple. You are given a set of rules and a scenario (e.g., Sihwale, Jane Doe, Pat) and must perform unifications to output a suggested investment.

Given Knowledge Base (Rules 1-8)	
1	savings_account(inadequate) → investment(savings).
2	savings_account(adequate) ∧ income(adequate) → investment(stocks).
3	savings_account(adequate) ∧ income(inadequate) → investment(combination).
4	∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ greater(X, minsavings(Y))) → savings_account(adequate).
5	∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minsavings(Y))) → savings_account(inadequate).
6	∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ greater(X, minincome(Y))) → income(adequate).
7	∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minincome(Y))) → income(inadequate).
8	∀X earnings(X, unsteady) → income(inadequate).

The Sihwale Problem (2023 Exam - 6 marks)
*Scenario: Sihwale has 5 dependents, an unsteady income of R45,000, and R25,500 in savings. Equations: minsavings(X) = 5000 * X. minincome(X) = 15000 + (5000 * X).*

Step 1: Write facts into predicates.
dependents(5).
amount_saved(25500).
earnings(45000, unsteady).

Step 2: Calculate minsavings and minincome.
minsavings(5) = 5000 * 5 = 25000.
minincome(5) = 15000 + (5000 * 5) = 40000.

Step 3: Evaluate Savings (Rules 4 & 5).
Is amount_saved > minsavings? 25500 > 25000. Yes.
This unifies with Rule 4 (greater). Therefore: **savings_account(adequate)**.

Step 4: Evaluate Income (Rules 6, 7, 8).
Sihwale's income is *unsteady*. This directly unifies with Rule 8: `∀X earnings(X, unsteady) → income(inadequate)`.
Therefore: **income(inadequate)**.

Step 5: Determine Final Investment (Rules 1, 2, 3).
We have `savings_account(adequate)` AND `income(inadequate)`.
This unifies with **Rule 3**.
Conclusion: `investment(combination)`.

Chapter 3: State Space Search Strategies

Search is the core of AI problem solving. A state space represents all possible configurations of a problem, linked by legal moves (operators). To search it, we must choose a direction and a strategy.

1. Search Direction: Goal-Driven vs. Data-Driven

A frequent exam question (e.g., 2024 Exam Q1.3, 2026 Test 1) asks you to recommend and justify a search direction based on problem characteristics.

Data-Driven (Forward Search)	Goal-Driven (Backward Search)
Starts from given facts/initial state and applies rules to see where they lead.	Starts from the desired goal and works backwards to find supporting data.
Use when: All data is given initially, there are many possible goals, or it's hard to form a hypothesis.	**Use when:** A goal is clearly formulated, problem data is not given (must be queried), or rules have many conditions but few conclusions.
Examples: Interpreting sonar readings, classifying plant species from attributes, bottom-up parsing.	*Examples:* Medical diagnosis (starting with a disease hypothesis), math proofs, automobile mechanical troubleshooting.

2. Uninformed Search Strategies: DFS vs. BFS

If you must search a tree without heuristics, you choose between Depth-First (DFS) and Breadth-First Search (BFS).

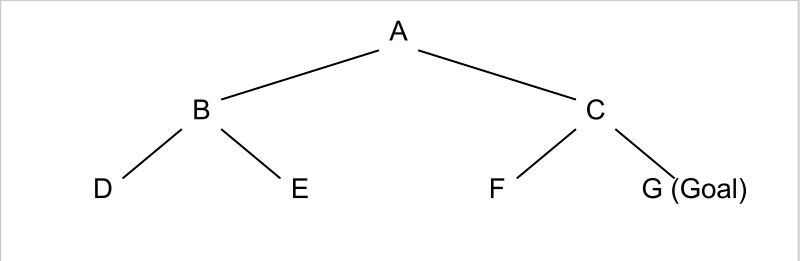
Breadth-First Search (BFS)	Depth-First Search (DFS)
Explores all nodes at level n before moving to level $n+1$.	Dives down a single path to the deepest leaf before backtracking.
Implemented using a **Queue** (First-In, First-Out).	Implemented using a **Stack** (Last-In, First-Out).
Guaranteed Optimality: Will always find the shortest path to a goal first.	**Not Optimal:** May find a deeper, sub-optimal goal first.
Memory Heavy: Must store entire levels of the tree.	**Memory Efficient:** Only stores the current path and immediate siblings.

Exam Application (Tic-Tac-Toe): For a simple game like Tic-Tac-Toe, **DFS** is preferred because the depth of the search tree is strictly bounded (maximum 9 moves), preventing infinite loops, and DFS uses significantly less memory than BFS, which would explode combinatorially at each level.

Chapter 3: Backtracking Search Trace

Dr. Baloyi requires you to *"Hand run the backtracking algorithm... keeping track of necessary data structures."* You must use three lists:

- **State List (SL):** The current path from the root to the current node (functions as a stack).
- **New State List (NSL):** Nodes waiting to be evaluated. When a node is expanded, its children are prepended (added to the front) of NSL.
- **Dead Ends (DE):** Nodes that have no children and are not the goal, or paths that have been fully explored and failed.



Example Trace: Find Goal G starting from A. (Left-to-right processing)

Initialize:
SL = [A]
NSL = [A]
DE = []

Iteration 1: Evaluate A. Not Goal. Generate children [B, C]. Prepend to NSL.
SL = [B, A]
NSL = [B, C, A]
DE = []

Iteration 2: Evaluate B. Not Goal. Generate children [D, E].
SL = [D, B, A]
NSL = [D, E, C, A]
DE = []

Iteration 3: Evaluate D. Not Goal. No children. Backtrack!
Move D to DE. Pop D from SL. Next in NSL is E.
SL = [E, B, A]
NSL = [E, C, A]
DE = [D]

Iteration 4: Evaluate E. Not Goal. No children. Backtrack!
Move E to DE. Pop E from SL. Pop B from SL (since all B's children failed).
SL = [C, A]
NSL = [C, A]
DE = [E, D, B]

Iteration 5: Evaluate C. Not Goal. Generate children [F, G].
SL = [F, C, A]
NSL = [F, G, A]
DE = [E, D, B]

Iteration 6: Evaluate F (Dead End). Backtrack to G. Evaluate G. GOAL!
Final SL (Path): [G, C, A]

Chapter 4: Heuristic Search Theory

A heuristic is a rule of thumb or an educated guess that helps reduce the search space in complex problems. They are *fallible*, meaning they might not always point to the optimal path, but they are useful when exact algorithms (like BFS/DFS) face combinatorial explosion.

Key Heuristic Properties (Quiz 01 & Exam focus)

- **Admissibility:** A heuristic is admissible if it *never overestimates* the true cost to reach the goal. If an admissible heuristic is used with the A* algorithm, it is mathematically guaranteed to find the shortest/optimal path.
- **Monotonicity:** A heuristic is monotonic if it is locally consistent. As you move along a path, the heuristic value never increases. (It satisfies the triangle inequality).
- **Informedness:** If heuristic H1 expands fewer nodes than H2, H1 is said to be *more informed*.

Hill Climbing

Hill climbing is a greedy local search. It expands the current state, looks at the immediate neighbors, and moves to the neighbor with the best heuristic value. It never keeps a list of unexplored nodes (no backtracking queue), so it uses very little memory.

The Three Pitfalls of Hill Climbing:

1. **Local Maxima:** Reaching a state that is better than all its immediate neighbors, but is not the global goal. The algorithm gets "stuck."
2. **Plateaus:** A flat area of the search space where all neighbors have the exact same heuristic value, giving the algorithm no direction on where to move.
3. **Ridges:** A steep slope where the highest point is not directly accessible by a single horizontal or vertical move (the neighbors appear worse, but stepping diagonally would improve).

Chapter 4: Best-First Search (A*) Trace

Best-First search improves on BFS and DFS by ordering the New State List (NSL / Open list) based on a heuristic evaluation function. The A* algorithm uses the formula: `f(n) = g(n) + h(n)`, where `g(n)` is the actual cost from start to node *n*, and `h(n)` is the estimated cost from *n* to the goal.

Tracing Best-First Search
Unlike backtracking which prepends to NSL (LIFO), Best-First Search maintains the open list as a **Priority Queue**, sorting it by the lowest `f(n)` value on every iteration.

Assume a graph where A is start. Heuristics h(n): A=5, B=4, C=3, D=2, E=4, Goal=0. Edge costs are 1.

Init: Open = [A(5)] *(Since g(A)=0, f(A) = 0 + 5)*. Closed = []

Iteration 1:
Pop A. Generate children B and C.
Calculate f(B) = g(A→B) + h(B) = 1 + 4 = 5.
Calculate f(C) = g(A→C) + h(C) = 1 + 3 = 4.
Sort Open by lowest f(n): Open = [C(4), B(5)].
Closed = [A]

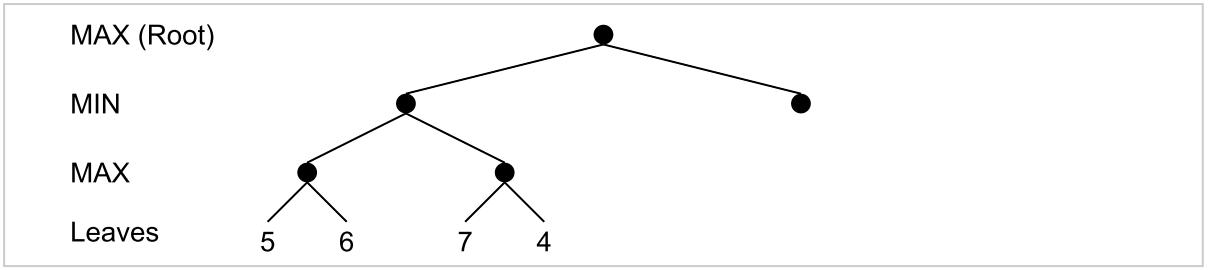
Iteration 2:
Pop C (lowest cost). Generate children D and Goal.
Calculate f(D) = g(A→C→D) + h(D) = 2 + 2 = 4.
Calculate f(Goal) = g(A→C→Goal) + h(Goal) = 2 + 0 = 2.
Insert into Open and sort: Open = [Goal(2), D(4), B(5)].
Closed = [A, C]

Iteration 3:
Pop Goal. Search terminates.
Path Found: A → C → Goal.

Critical Exam Note: Always write down the Open List in sorted order at every step. If two nodes have the same `f(n)`, you can break ties alphabetically or arbitrarily, but the sort order is where the marks are awarded.

Chapter 4: Games & Alpha-Beta Pruning

In adversarial search (like Chess or Checkers), we use the **Minimax** algorithm. One player (MAX) tries to maximize the score, while the opponent (MIN) tries to minimize it. **Alpha-Beta Pruning** is an optimization technique that stops evaluating a branch when it finds a move that is worse than a previously examined move. Its primary benefit is that it dramatically reduces the number of nodes evaluated without affecting the final outcome.



How to trace Alpha-Beta Pruning (2024 Exam)

1. Traverse down the far-left path to the leaves. Evaluate the first MAX node: `max(5, 6) = 6`. This MAX node returns 6 to the MIN node above it.
2. The MIN node now knows it can get a score of *at most* 6 from its left branch. (Beta = 6).
3. Traverse down MIN's right branch to the next MAX node. Look at its first leaf: `7`. MAX will choose at least 7.
4. **PRUNE:** Since the right MAX node guarantees a score of ≥ 7, but the MIN node above it already has a choice of 6 (and wants the lowest number), MIN will *never* choose the right branch. We prune the remaining leaves of the right branch (e.g., the `4` in the diagram is never evaluated).

Exam tip: Draw a slash through pruned branches on the diagram.

Chapter 6: Production Systems Architecture

A production system is an architecture for problem solving that uses rules to modify a central database of facts. It is the core engine behind many Expert Systems. (Tested heavily in 2024).

The 3 Components of a Production System

1. **Production Rules:** The "knowledge" of the system. Each rule consists of a **Condition** (the pattern to match against current facts) and an **Action** (what to do or add if the condition is met).
2. **Working Memory:** The "global database" of current facts. This contains the initial state, intermediate findings, and the current goal.
3. **Recognize-Act Cycle:** The control structure. It matches the Condition of rules against Working Memory. It identifies all valid rules (forming the *Conflict Set*), selects one rule to fire (using conflict resolution), and executes its Action, modifying Working Memory. This repeats until a goal is reached.

Conflict Resolution Strategies

When multiple rules match the Working Memory simultaneously, the system must decide which single rule to execute.

- **Refraction:** A rule that has just fired cannot fire again on the exact same data. (Prevents infinite loops).
- **Recency:** Prefers rules that match the data most recently added to working memory. (Keeps the system focused on the current line of reasoning).
- **Specificity:** Prefers rules that have more specific conditions (e.g., a rule with 3 conditions beats a rule with 1 condition).

The Blackboard Architecture
Used for highly complex problems (like speech recognition) where different types of expertise are needed.

- **Knowledge Sources (KS):** Independent specialist modules (e.g., one KS for grammar, one for phonetics).
- **The Blackboard:** A shared global database where KSs post their partial solutions.
- **Control Shell:** Coordinates which KS gets to read/write to the blackboard next.

Chapter 6: Chaining & Pattern Search Traces

This chapter demands practical diagramming. Dr. Baloyi tests both Forward Chaining rule firing (2024 Exam) and the Knight's Tour trace.

Forward Chaining Trace (8 marks, 2024 Exam)
Rules given:
1. P ∧ Q → goal | 2. R ∧ S → P | 3. X ∧ R → Q
4. K ∧ U → Q | 5. Z → S | 6. Start → Z ∧ R ∧ X

Trace Table (Data-Driven):

Cycle	Working Memory	Conflict Set (Matched Rules)	Rule Fired
0	{ Start }	{ Rule 6 }	Rule 6
1	{ Start, Z, R, X }	{ Rule 5, Rule 3 }	Rule 5
2	{ Start, Z, R, X, S }	{ Rule 3, Rule 2 }	Rule 2
3	{ Start, Z, R, X, S, P }	{ Rule 3 }	Rule 3
4	{ Start, Z, R, X, S, P, Q }	{ Rule 1 }	Rule 1
5	{ Start, Z, R, X, S, P, Q, goal }	Empty (Goal Reached)	Halt

Note: The diagram for this is a tree branching from Start → Z,R,X. Then Z → S. Then (R,S) converge to P. (X,R) converge to Q. Finally (P,Q) converge to goal.

Knight's Tour: pattern_search on path(6,9)
Given a 3x3 chessboard, trace the rules to move from square 6 to square 9. Legal moves for square 6 are to 1 and 7. Legal moves for 1 are 6, 8. Legal moves for 7 are 2, 6. Legal moves for 8 are 1, 3. Legal moves for 2 are 7, 9.

The recursive `pattern_search` expands the current square to all valid neighbors, checking if one is the goal.
Trace:
Current State: 6. Goal: 9.
1. Apply rule for 6: Generates states [1, 7]. Not goal. Push to Working Memory.
2. Evaluate state 1: Generates [6, 8]. (Discard 6 due to cycle). Next state is 8.
3. Evaluate state 7: Generates [2, 6]. (Discard 6). Next state is 2.
4. Evaluate state 8: Generates [3]. Not goal.
5. Evaluate state 2: Generates [7, 9]. **9 is the GOAL!**
Path: 6 → 7 → 2 → 9.

Chapter 7: Knowledge Representation (Frames & Scripts)

This chapter moves beyond logic into structured representation. Issues in KR include **Exhaustiveness** (the ability of a language to capture all necessary domain facts) and **Computational Efficiency** (how fast the system can reason over those facts).

Frames

A frame is a data structure representing a stereotypical entity (like a class in object-oriented programming). It contains **Slots** that hold attributes, values, or pointers to other frames (Inheritance).

- **Inheritance Slot:** *"The 'hotel phone' might be a special instance of 'phone'..."* This represents an **IS-A** or Specialization slot. (Tested 2024)
- **Procedural Attachment (Demons):** Slots can contain code instead of static values. A `when-needed` demon calculates a value on the fly (e.g., Age derived from Birthdate). A `when-changed` demon updates dependent fields if a value is altered.

Scripts (Tested heavily 2024 & 2026)
A script is an extension of a frame used to represent a stereotypical sequence of events in time (e.g., going to a restaurant). The components you MUST cover are:

1. **Entry Conditions:** Prerequisites that must be true for the script to occur (e.g., Customer is hungry, Customer has money).
2. **Props:** The physical objects involved in the script (e.g., Tables, Menu, Food, Bill).
3. **Roles:** The agents or actors involved (e.g., Customer, Waiter, Cook, Cashier).
4. **Scenes:** The sequence of events (e.g., Scene 1: Entering, Scene 2: Ordering, Scene 3: Eating, Scene 4: Exiting).
5. **Results (Exit Conditions):** The state of the world after the script ends (e.g., Customer is full, Customer has less money, Owner has more money).

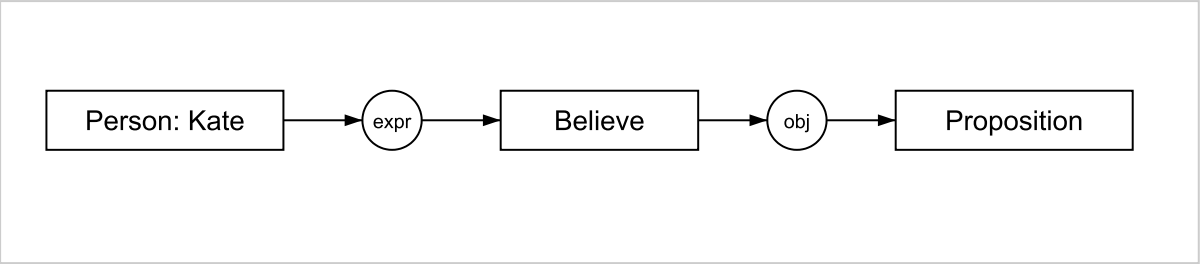
Exam Tip: Writing Custom Scripts
You may be asked to define a script for a new situation, like **"writing an examination"**. Do not use generic restaurant props!
Entry Conditions: Student is registered, Exam date is today.
Props: Exam script, Pen, ID card, Desk.
Roles: Student, Invigilator, Examiner.
Scenes: Enter venue → Hand out papers → Write → Submit → Leave.
Results: Student is tired, Invigilator has scripts.

Chapter 7: Conceptual Graphs

Conceptual graphs (CG) are a visual network for representing meaning. They use two main node types: **Concept nodes** (drawn as boxes, representing entities, acts, or states) and **Conceptual Relation nodes** (drawn as circles, indicating how concepts are linked).

Experiencer Relation vs. Experiencer Link (2026 Test 2 Q2.1):

- **Experiencer Relation:** A circle node in the graph (e.g., `(expr)`) that formally binds an act of perception/emotion to the agent experiencing it.
- **Experiencer Link:** The directed arc (arrow) that physically connects the concept node (e.g., `[Person: Kate]`) to the relation node `(expr)`, and then connects `(expr)` to the mental act `[Believe]`.

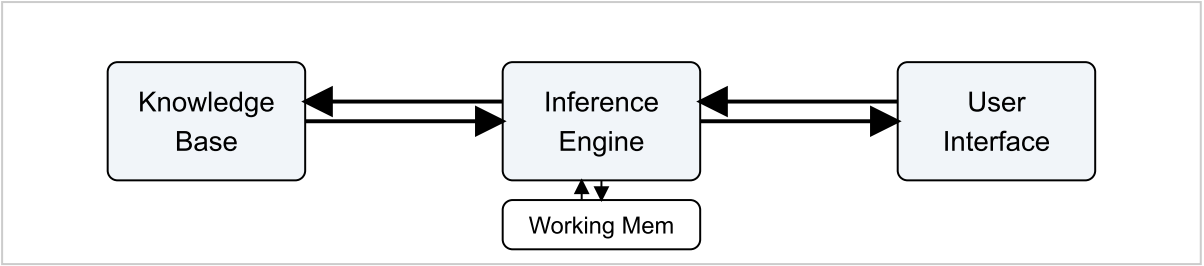


Translation: Kate believes a proposition.

> **Translating Sentences to CGs**
> If you are given a sentence like *"Kate believes that John likes pizza"*, you construct the graph hierarchically.
> 1. The outer graph represents Kate's belief: `[Person: Kate] → (expr) → [Believe] → (obj) → [Proposition]`.
> 2. The inner graph (inside the Proposition box) represents the nested sentence: `[Person: John] → (expr) → [Like] → (obj) → [Food: Pizza]`.

Chapter 8: Strong Method Problem Solving (Expert Systems)

Strong methods use highly domain-specific knowledge (unlike DFS/BFS which are weak, general methods). Expert systems are the prime example. A core exam question (2026 Test 2) asks for the architecture and appropriateness guidelines of an expert system.



Architecture of a Typical Expert System (6 marks)

6 Guidelines for Expert System Appropriateness (2026 Test 2 - 6 marks)
A problem is appropriate for an expert system if:

1. The need for a solution justifies the high cost and effort of building the system.
2. Human expertise is scarce, or expert knowledge is being lost (e.g., retirement).
3. The problem can be solved by human experts, indicating it is well-understood.
4. The problem domain is narrow and well-defined (not broad common-sense reasoning).
5. The problem requires cognitive skills and heuristics, not just raw mathematical calculation.
6. The system will be used in an environment where it can safely operate and its logic can be explained to users.

Chapter 8: Case-Based Reasoning & Planning

In addition to rule-based systems, Chapter 8 introduces Case-Based and Model-Based reasoning, as well as AI Planning algorithms (STRIPS / Blocks World).

Rule-Based vs. Case-Based Reasoning

Rule-Based Reasoning	Case-Based Reasoning (CBR)
Uses IF-THEN rules extracted from experts.	Uses a database of past historical cases to solve new problems.
Requires deep analysis of domain logic to build the knowledge base.	Retrieves the most similar past case, adapts it, and saves the new result.
Disadvantage: Brittle; fails entirely if a scenario doesn't match any rule.	**Disadvantage:** Requires a massive, well-indexed database; adaptation logic can be very difficult to write.

Planning: Blocks World Operators (STRIPS)
AI Planners must define states using predicates, and operators using Precondition (P), Add (A), and Delete (D) lists. You must memorize these four fundamental operators for the Blocks World domain:

1. pickup(A):
P: ontable(A) ∧ clear(A) ∧ handempty
D: ontable(A), clear(A), handempty
A: holding(A)

2. putdown(A):
P: holding(A)
D: holding(A)
A: ontable(A), clear(A), handempty

3. stack(A, B):
P: holding(A) ∧ clear(B)
D: holding(A), clear(B)
A: on(A, B), clear(A), handempty

4. unstack(A, B):
P: on(A, B) ∧ clear(A) ∧ handempty
D: on(A, B), clear(A), handempty
A: holding(A), clear(B)

End of Comprehensive Guide. Master these structures and traces to secure full marks on the SCOA032 assessments.